CSE 4304-Data Structures Lab. Winter 2024-25
Date: 04 February 2026
Target Group: All groups
Topic: Basic Tasks on Disjoint Set, Graph Data Structures.

Instructions:

- Task naming format: fullID_T01L01_2A.c/cpp
- If you find any issues in the problem description/test cases, comment in the Google Classroom.
- If you find any tricky test cases that I didn't include but that others might forget to handle, please comment! I'll be happy to add them.
- Use appropriate comments in your code. This will help you recall the solution more easily in the future.
- The obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library.
- Modified sections will be marked with BLUE color.
- You are allowed to use the STL stack unless it's specifically mentioned to use manual functions.

Group	**Tasks**
1A/1B/2A/2B	1 2 3 4
Assignment	

Task 1: Basic operations of DSU

Consider that N items are stored in a Disjoint Set data structure, identified as 0, 1, 2, …, N-1. There can be three types of operations conducted on the items:

1. **initialization**: creates separate sets for each item, with each item pointing to itself as its representative.
2. **find(item)**: finds the disjoint set containing that item and returns its representative.
3. **union(i,j)**: connects the representatives of two sets and forms a new set by adding the second representative to the first one (doesn't consider rank)

After each input, print the state of the 'representative array', where every i-th item contains its representative information.

Sample Input	**Sample Output**	**Remarks**
10		
1	0 1 2 3 4 5 6 7 8 9	Initialize
2 3	1	
3 1 3	0 1 2 1 4 5 6 7 8 9	
2 1	1	
2 3	1	r(3) = 1, previously 3
3 3 5	0 1 2 1 4 1 6 7 8 9	
3 5 7	0 1 2 1 4 1 6 1 8 9	r(5)=1, r(7)=7. So 7 is connected to 1 instead of 5. Only representatives are connected, not items.
3 6 8	0 1 2 1 4 1 6 1 6 9	
3 8 9	0 1 2 1 4 1 6 1 6 6	
3 4 8	0 1 2 1 4 1 4 1 6 6	Attention: r(4)=4, r(8)=6. So connecting 4,8 implies connecting 4,6. So r(6) is updated to 4
2 8	4	
2 6	4	
2 4	4	
3 5 6	0 1 2 1 1 4 1 6 6	Think: Why is r(6) not updated, rather r(4)?
2 8	1	find(8)=6, so call find(6). find(6)=4, so call find(4) find(4)=1, so call find(1) find(1)=1. As input=output, this is the stopping condition. We return 1. (Path compression can help us in this situation, but in this task, we haven't used it)

Here, r(item) means representative of an item.

Task 2: DSU with Union-By-Rank and Path Compression

Improve the Union(i,j) function of Task1 by introducing the ‘Union by Rank’ approach, where the representative of i will be connected with the representative of j, if it has a lesser or equal rank.
Print the rank of each node after each union operation. Besides, optimize the Find function using the ‘path compression technique’.

- Similar to Task 1, the options 1, 2, and 3 will be dedicated to initialization, find, and union operations, respectively.
- To reflect the impact of the path compression strategy, now we’ll print some extra information.
- Introduce a new display feature that will be initiated if Option 4 is given.

Sample Input	**Sample Output**	**Remarks**
10 1	0(0) 1(0) 2(0) 3(0) 4(0) 5(0) 6(0) 7(0) 8(0) 9(0)	Initialization: each item is part of a separate set, rank is 0.
3 1 3	0(0) **3**(0) 2(0) 3(**1**) 4(0) 5(0) 6(0) 7(0) 8(0) 9(0)	1 & 3 both have equal ranks. As per our given slide: if (rank[ri] <= rank[rj]) parent[ri] = rj Hence, r(1) is updated to 3. Previously, r(3) was updated to 1. This logic depends on implementation. We followed this logic throughout this simulation. Note: The rank of 3 is increased.
2 1	**3** **f(1) f(3)**	Now our find function prints the function calls for better understanding
2 3	**3** **f(3)**	As f(3) = 3, we stop in the first step.
3 3 5	0(0) 3(0) 2(0) 3(1) 4(0) **3**(0) 6(0) 7(0) 8(0) 9(0)	5 has a lower rank than 3. r(5) is connected with r(3). Now, r(3)=3 But r(3) is not increased. Rank only increases when two equal rank sets are connected.
3 5 7	0(0) 3(0) 2(0) 3(1) 4(0) 3(0) 6(0) **3**(0) 8(0) 9(0)	r(5) is 3, r(7) is 7 rank(3) > rank(7) [1>0] Hence, 7 is connected with 3.
3 6 8	0(0) 3(0) 2(0) 3(1) 4(0) 3(0) **8**(0) 3(0) 8(**1**) 9(0)	
3 8 9	0(0) 3(0) 2(0) 3(1) 4(0) 3(0) 8(0) 3(0) 8(1) **8**(0)	
3 4 8	0(0) 3(0) 2(0) 3(1) **8**(0) 3(0) 8(0) 3(0) 8(1) 8(0)	
2 8	**8** **f(8)**	
2 6	8 f(6) f(8)	
2 4	8	

	f(4) f(8)	
3 5 6	0(0) 3(0) 2(0) **8**(1) 8(0) 3(0) 8(0) 3(0) 8(**2**) 8(0)	
2 8	8 f(8)	
2 3	8 f(3) f(8)	
2 7	**8** f(7) f(3) f(8)	This path will get compressed in this find call 7 will now be directly connected to 8
4	0(0) 3(0) 2(0) 8(1) 8(0) 3(0) 8(0) **8**(0) 8(2) 8(0)	

Note: Once the path is compressed, as per the definition of rank, it might be reduced. We did not handle this issue here. If you can properly simulate this with code, you can get a bonus mark. (That can even be included in a slide in the future. If you think your code+simulation is good enough, pls Email me. I'll consider adding it.)

Task 3: Represent Graph using Adjacency List & Adjacency Matrix

You are given a graph with V vertices and E edges. Your task is to represent the graph in two different ways:

- Adjacency List: This is a list of vertices where each vertex points to a list of vertices that are connected to it by an edge.
- Adjacency Matrix: This is a 2D matrix V x V, where matrix[i][j] is 1 if there is an edge from vertex i to vertex j and 0 otherwise.

Then, display the following information:

- The Adjacency List representation of the graph.
- The Adjacency Matrix representation of the graph.

Input Format:

- The first line contains two integers V and E, where V is the number of vertices and E is the number of edges in the graph.
- The next E lines each contain two integers u and v, indicating an edge from vertex u to vertex v.

Sample Input	**Sample Output**
5 6 1 2 1 3 2 4 3 4 3 5 4 5	Adjacency List: 1: 2 3 2: 1 4 3: 1 4 5 4: 2 3 5 5: 3 4 Adjacency Matrix: 0 1 1 0 0 1 0 0 1 0 1 0 0 1 1 0 1 1 0 1 0 0 1 1 0
6 3 1 2 2 3 4 5	Adjacency List: 1: 2 2: 1 3 3: 2 4: 5 5: 4 6: Adjacency Matrix: 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0

Task 4: Topological Sort Using Queue Approach

You are given a Directed Acyclic Graph (DAG) with N nodes and M edges. Your task is to perform a topological sort using the Queue approach.

The Queue approach is based on Kahn's Algorithm for Topological Sorting. In this approach, we maintain a queue to store nodes with no incoming edges (i.e., their indegree is 0). We then remove these nodes from the graph and reduce the in-degree of their neighbors. This continues until there are no more nodes left in the queue.

Your task is to print the topologically sorted order of nodes. If it is not possible to complete the topological sorting due to a cycle in the graph, return an empty array.

Input Format:

- The first line contains two integers N (the number of nodes) and M (the number of edges).
- The next M lines contain two integers a and b representing a directed edge from node a to node b.

Output Format:

- Print the topologically sorted order of nodes in a single line, separated by spaces.
- If it is not possible to perform topological sorting due to a cycle, print an empty array [].

The graph is guaranteed to be valid (no multiple edges, no self-loops).

Sample Input	**Sample Output**	**Explanation**
6 7 3 4 3 5 0 1 0 2 4 5 1 3 2 3	0 1 2 3 4 5	The graph has multiple paths. Both [0, 1, 2, 3, 4, 5] and [0, 2, 1, 3, 4, 5] are valid topological sorts.
5 5 4 2 4 3 3 1 2 0 3 0	4 2 0 3 1	
4 4 0 1 0 2 1 3 2 3	0 1 2 3	Both [0, 1, 2, 3] and [0, 2, 1, 3] are valid topological sorts. The graph allows for two different valid orderings.
3 3 0 1 1 2 2 0	[ ]	The graph contains a cycle (0 → 1 → 2 → 0), which makes topological sorting impossible. The output is an empty array [].
6 3	0 1 2 3 4 5	The graph is disconnected, but the topological sorting

0 1 2 3 4 5		can still be done independently for each component. There can be multiple valid topological orderings like [0, 1, 2, 3, 4, 5] or [2, 3, 0, 1, 4, 5].
5 5 0 1 1 2 2 3 3 4 4 2	[ ]	The graph contains a cycle (2 → 3 → 4 → 2), so topological sorting is not possible.
6 6 0 1 0 2 1 3 2 3 3 4 3 5	0 1 2 3 4 5	This is a larger graph with multiple valid topological sorts, and both orders are acceptable as valid outputs.